Modelos de Calidad de Software

Escuela de Tecnología

Mg. Daniel Arias, **PMP™** y **Scaled Scrum Master™**

¿Qué aprendimos la clase pasada?

- Concepto prueba unitaria
- Introducción a JUNIT
- Anotaciones (After, Test, Before)
- Principales Asserts en JUNIT
- Creación y Ejecución de una prueba unitaria simple con Junit en NetBeans.

idat

¿Qué aprendimos la clase pasada?

- Segunda Evaluación Continua

idat

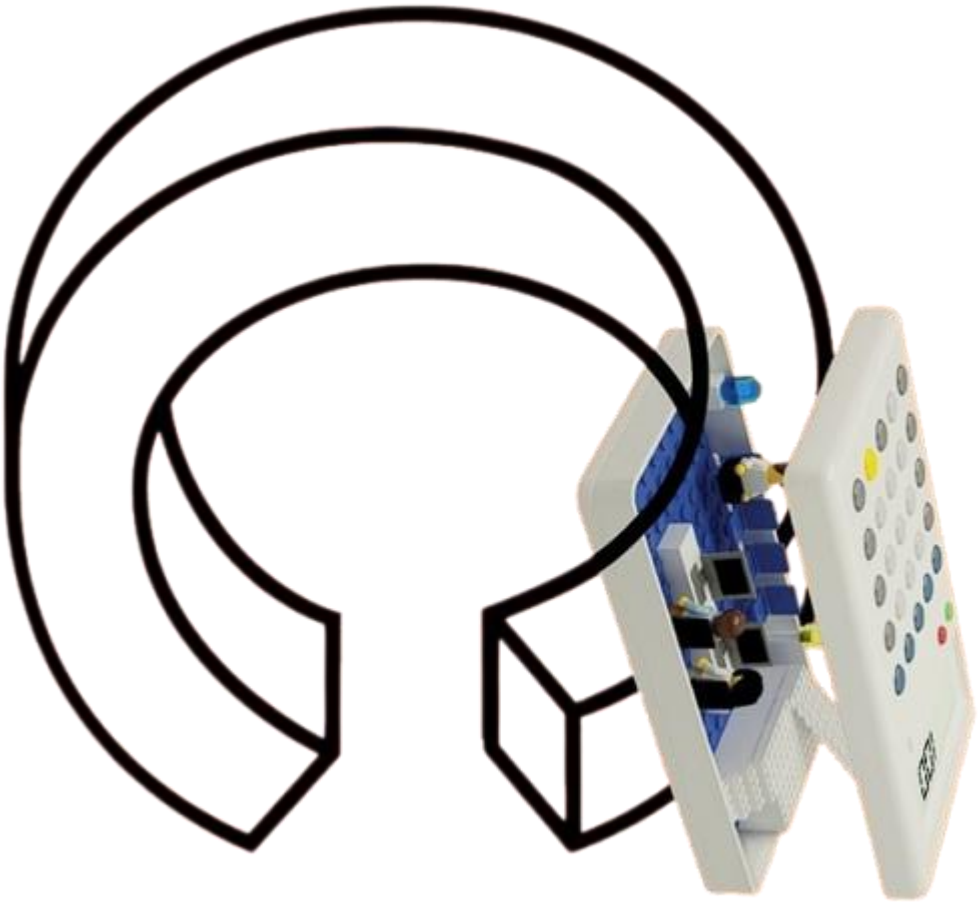
Conversemos

http://www.menti.com

idat

Pruebas Unitarias con el framework JUnit – parte 2

Semana 8



idat

Logro de aprendizaje

- Creación y Ejecución de una prueba unitaria utilizando las anotaciones y asserts del framework JUNIT

Actividad de aprendizaje

ACTIVIDAD VIRTUAL
Revisa y analiza el enlace sugerido en la plataforma virtual luego realiza la actividad propuesta

- Crear una prueba unitaria en la cual se incluya las anotaciones y asserts de JUNIT.

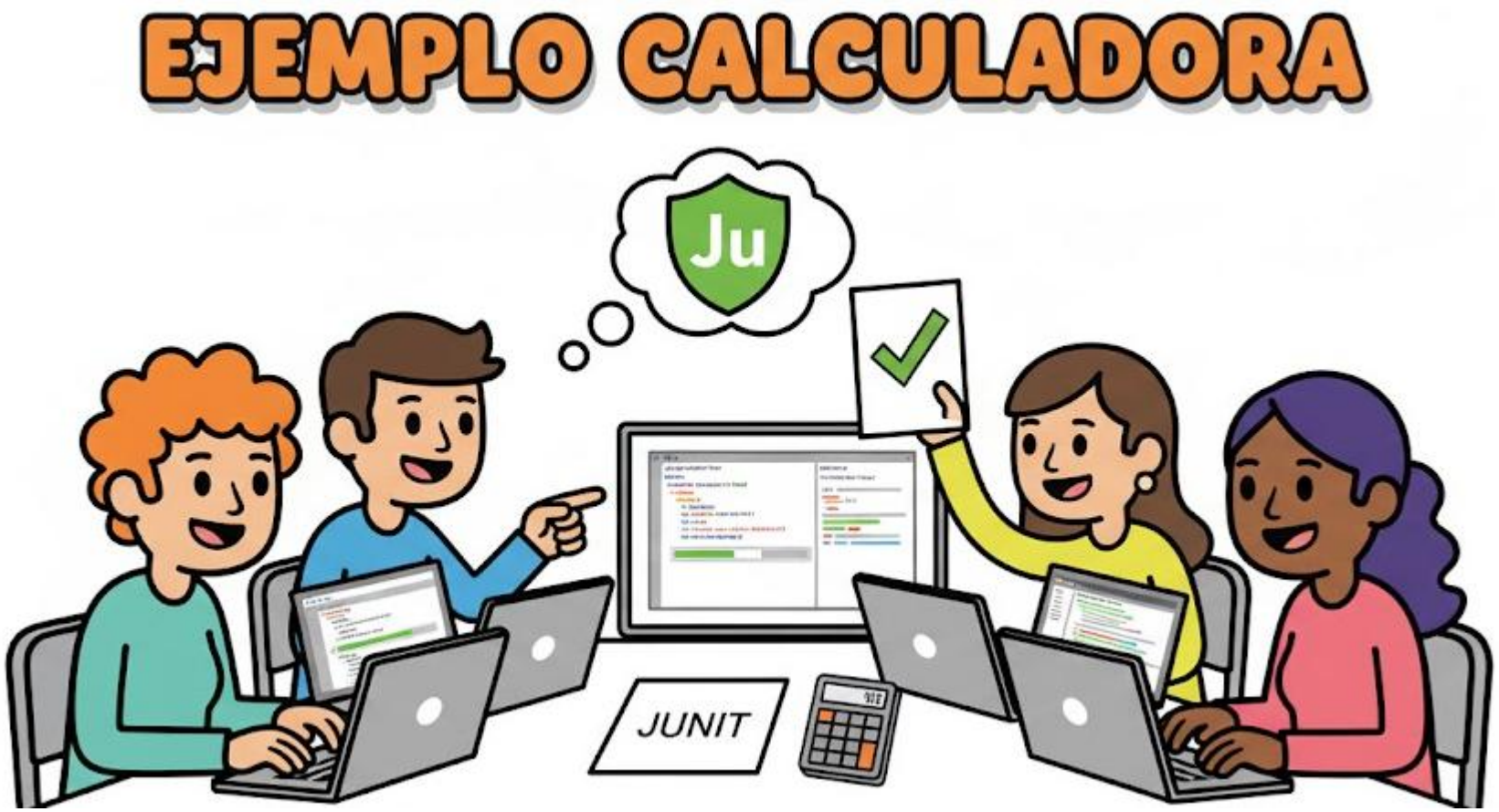
idat

Logro de aprendizaje

Construye pruebas unitarias intermedias usando todas las anotaciones y asserts de JUNIT.

idat

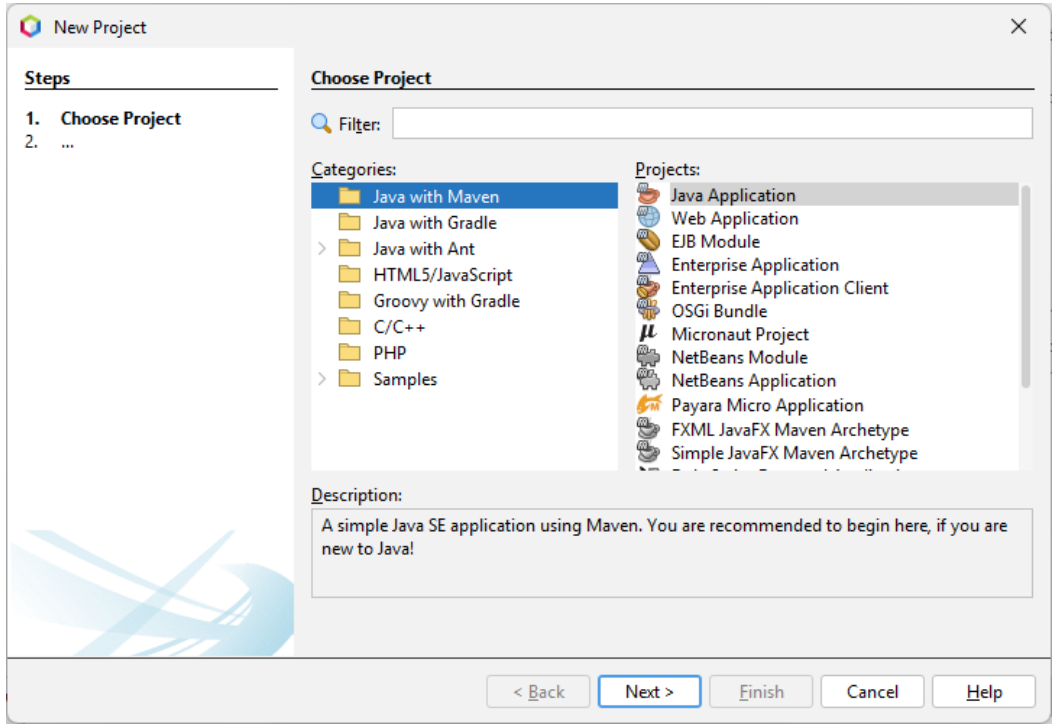
Laboratorio: JUnit

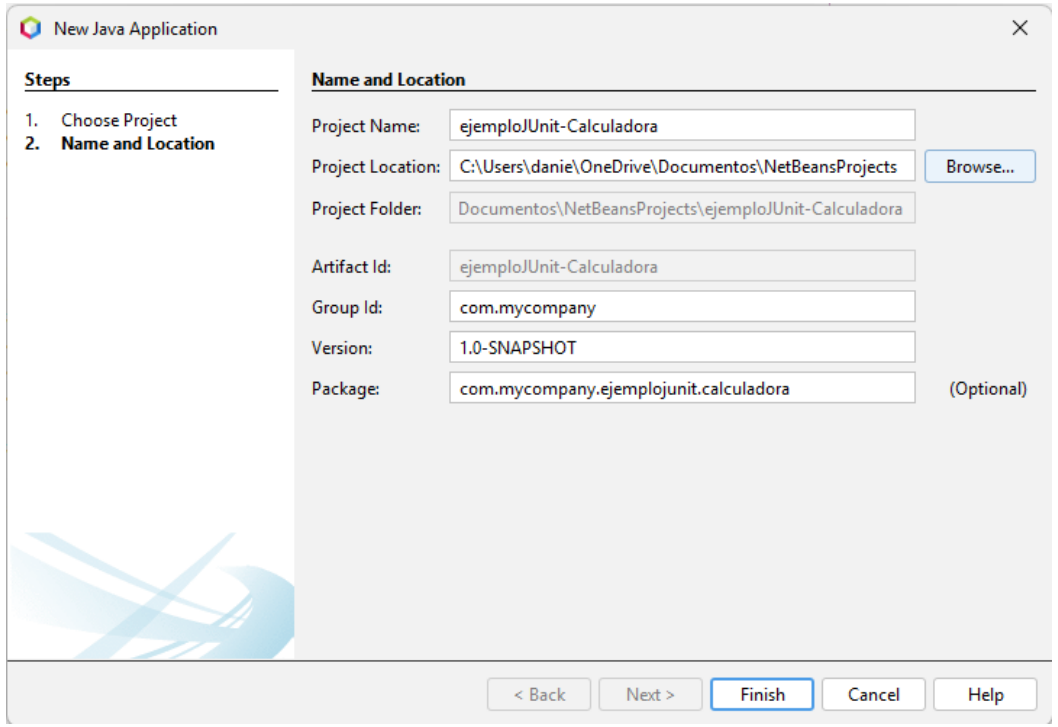


idat

Ejecutar Apache Netbeans

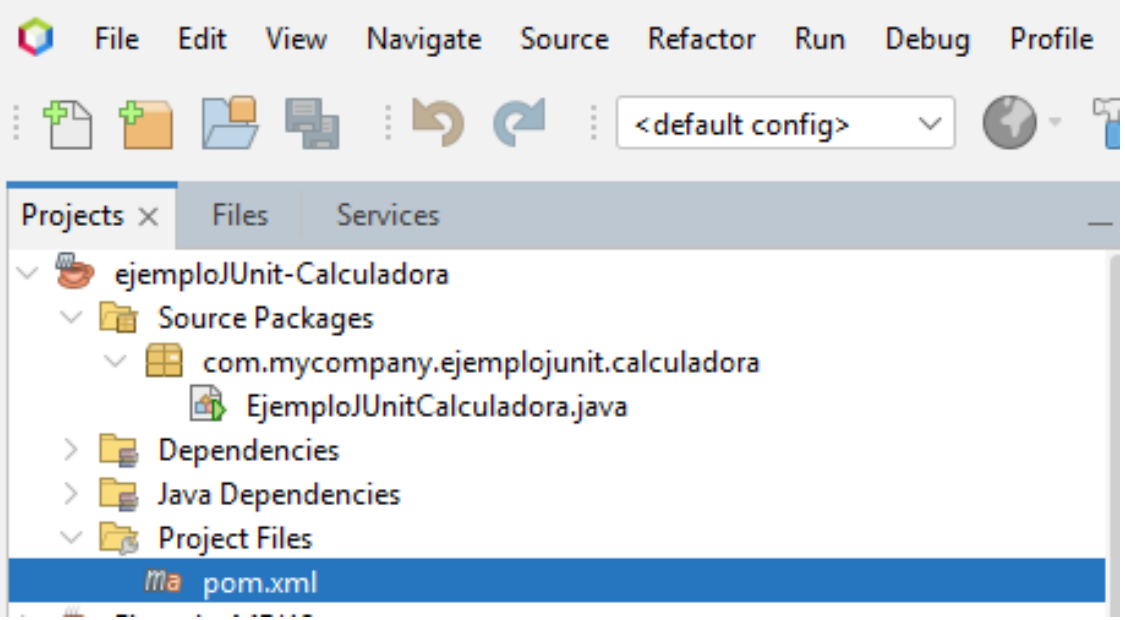
Creamos el Proyecto “ejemploJUnit-Calculadora”





idat

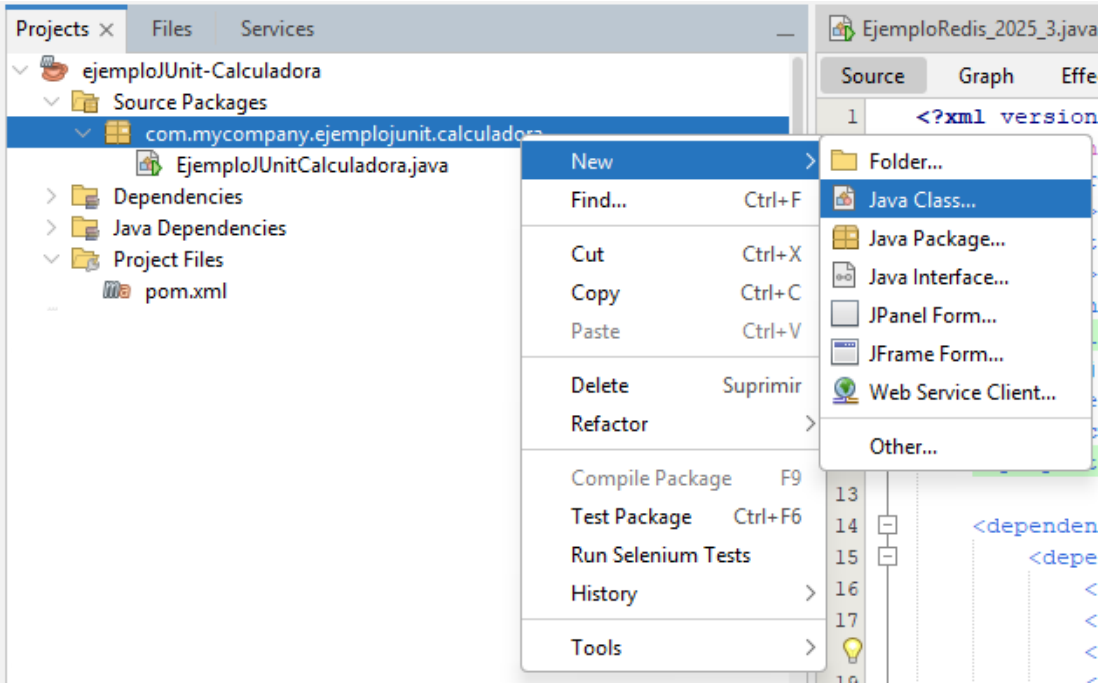
Modificar el pom.xml

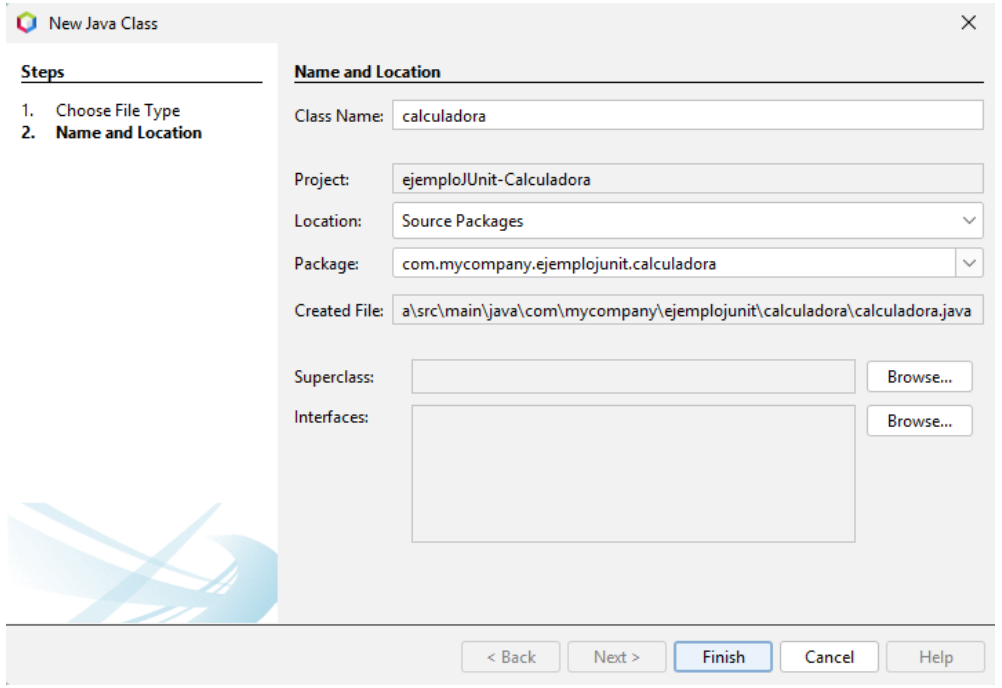



```
<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.14.0</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

idat

Creamos la Clase: calculadora





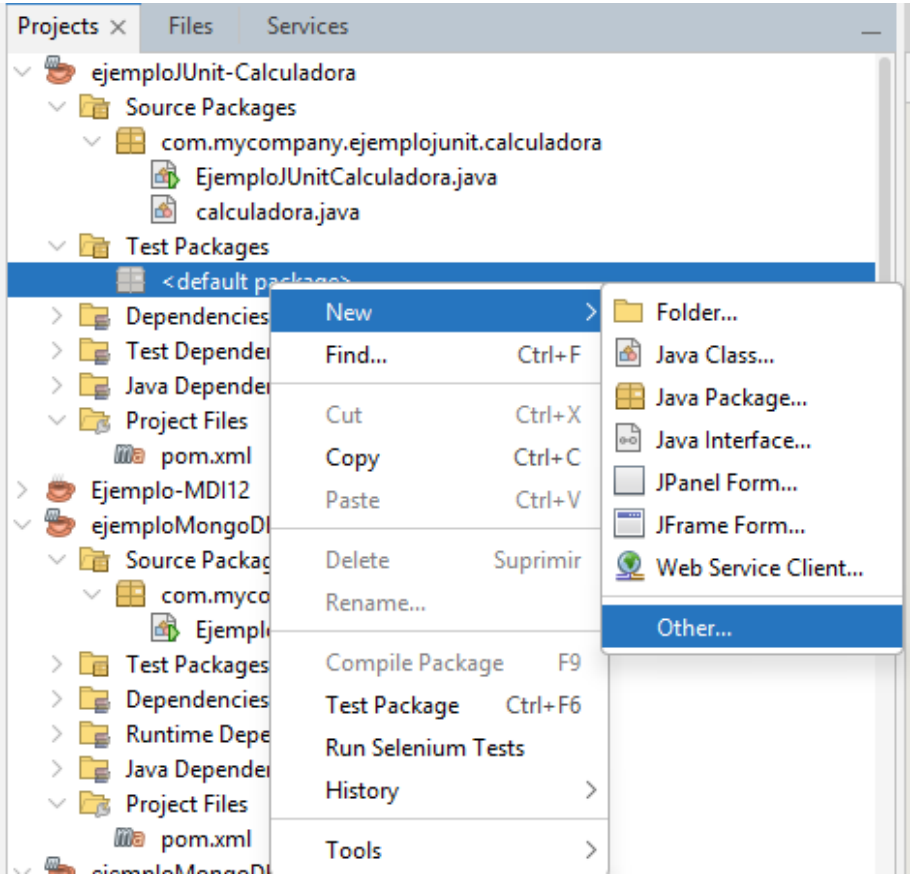
idat

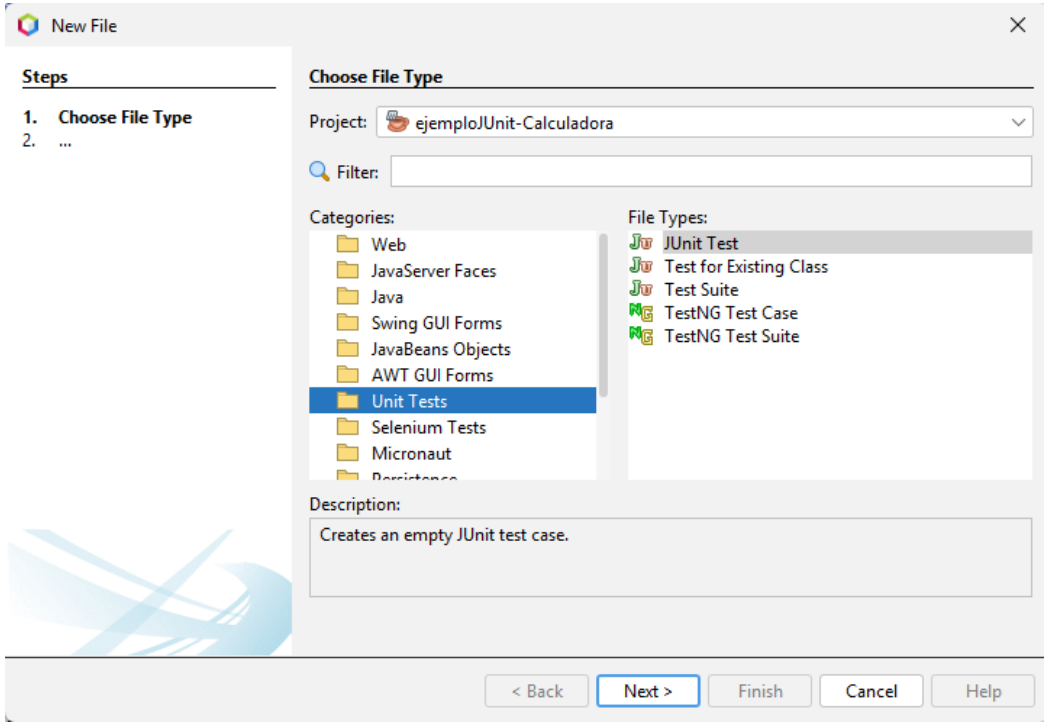
Codificamos la Clase: calculadora

```java
11  public class calculadora {
12
13      public double sumar(double a, double b) {
14          return a + b;
15      }
16
17      public double restar(double a, double b) {
18          return a - b;
19      }
20
21      public double multiplicar(double a, double b) {
22          return a * b;
23      }
24
25      public double dividir(double a, double b) {
26          if (b == 0) {
27              throw new ArithmeticException("No se puede dividir entre 0");
28          }
29
30          return a / b;
31      }
32
33      public boolean esPar(int numero) {
34          return numero % 2 == 0;
35      }
36
37      public double potencia(double base, double exponente) {
38          return Math.pow(base, exponente);
39      }
40  }
```

idat

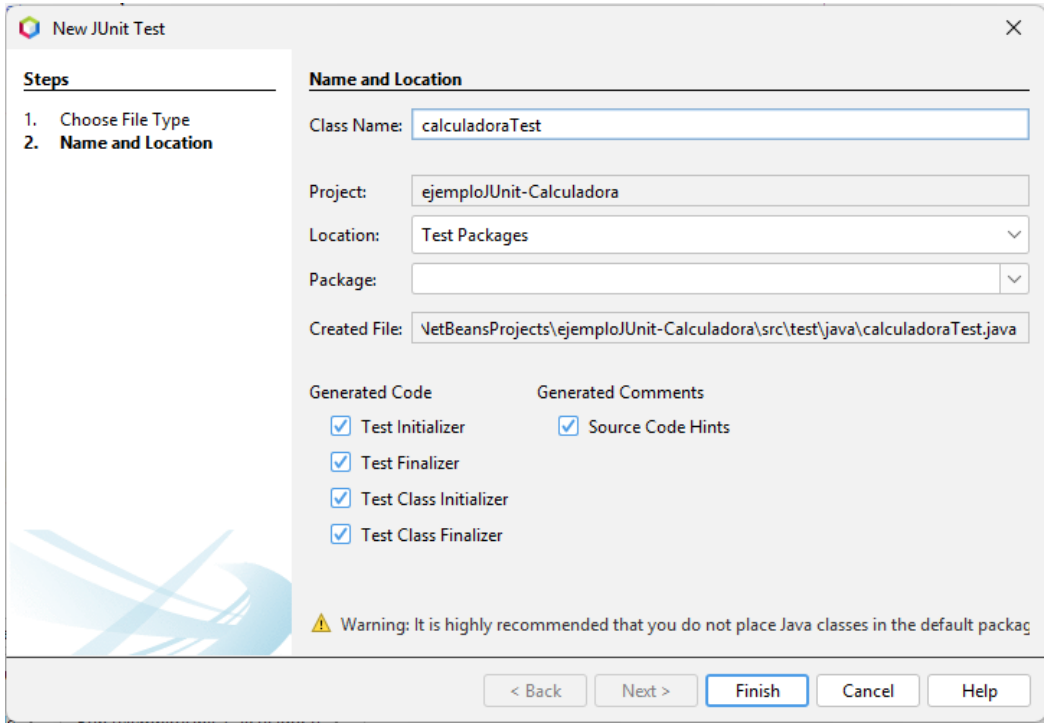
Creamos la Clase: calculadoraTest





idat

Codificamos la Clase: calculadoraTest




```
5
6   package com.mycompany.ejemplojunit.calculadora;
7
8   import org.junit.jupiter.api.BeforeEach;
9   import org.junit.jupiter.api.DisplayName;
10  import org.junit.jupiter.api.Test;
11  import static org.junit.jupiter.api.Assertions.*;
```

idat

Codificamos la Clase: calculadoraTest

```java
16  public class calculadoraTest {
17
18      private calculadora Calculadora;
19
20      @BeforeEach
21      public void setUp() {
22          Calculadora = new calculadora();
23      }
24
25      @Test
26      @DisplayName("Suma de dos números Positivos")
28      void testSumarPositivos() {
29          // Preparación (Arrange)
30          double a = 5.0;
31          double b = 3.0;
32
33          // Ejecución (Act)
34          double resultado = Calculadora.sumar(a, b);
35
36          // Verificación (Assert)
37          assertEquals(8.0, resultado, "5 + 3 debe ser 8");
38      }
39
40      @Test
41      @DisplayName("Resta de números")
42      void testRestar() {
43          assertEquals(2.0, Calculadora.restar(5, 3));
44          assertEquals(-2.0, Calculadora.restar(3, 5));
45      }
46
47      @Test
48      @DisplayName("Multiplicación de números")
49      void testMultiplicar() {
50          assertEquals(15.0, Calculadora.multiplicar(5, 3));
51          assertEquals(0.0, Calculadora.multiplicar(5, 0));
52      }
```

```java
53      @Test
54      @DisplayName("División válida")
55      void testDividirValido() {
56          assertEquals(2.0, Calculadora.dividir(6, 3));
57          assertEquals(2.5, Calculadora.dividir(5, 2));
58      }
59
60      @Test
61      @DisplayName("División por cero debe lanzar excepción")
62      void testDividirPorCero() {
63          // Verificar que se lanza la excepción esperada
64          ArithmeticException exception = assertThrows(ArithmeticException.class,
65              () -> Calculadora.dividir(5, 0));
66
67          assertEquals("No se puede dividir por cero", exception.getMessage());
68      }
69
70      @Test
71      @DisplayName("Verificar números pares e impares")
72      void testEsPar() {
73          assertTrue(Calculadora.esPar(4));
74          assertTrue(Calculadora.esPar(0));
75          assertFalse(Calculadora.esPar(3));
76          assertFalse(Calculadora.esPar(7));
77      }
78
79      void testPotencia(double base, double exponente, double expected) {
80          assertEquals(expected, Calculadora.potencia(base, exponente), 0.001);
81      }
82
83      @Test
84      @DisplayName("Múltiples aserciones en un solo test")
85      void testMultiplesOperaciones() {
86          assertAll("Operaciones múltiples",
87              () -> assertEquals(8, Calculadora.sumar(5, 3)),
88              () -> assertEquals(2, Calculadora.restar(5, 3)),
89              () -> assertEquals(15, Calculadora.multiplicar(5, 3)),
90              () -> assertEquals(2, Calculadora.dividir(6, 3))
91          );
92      }
```

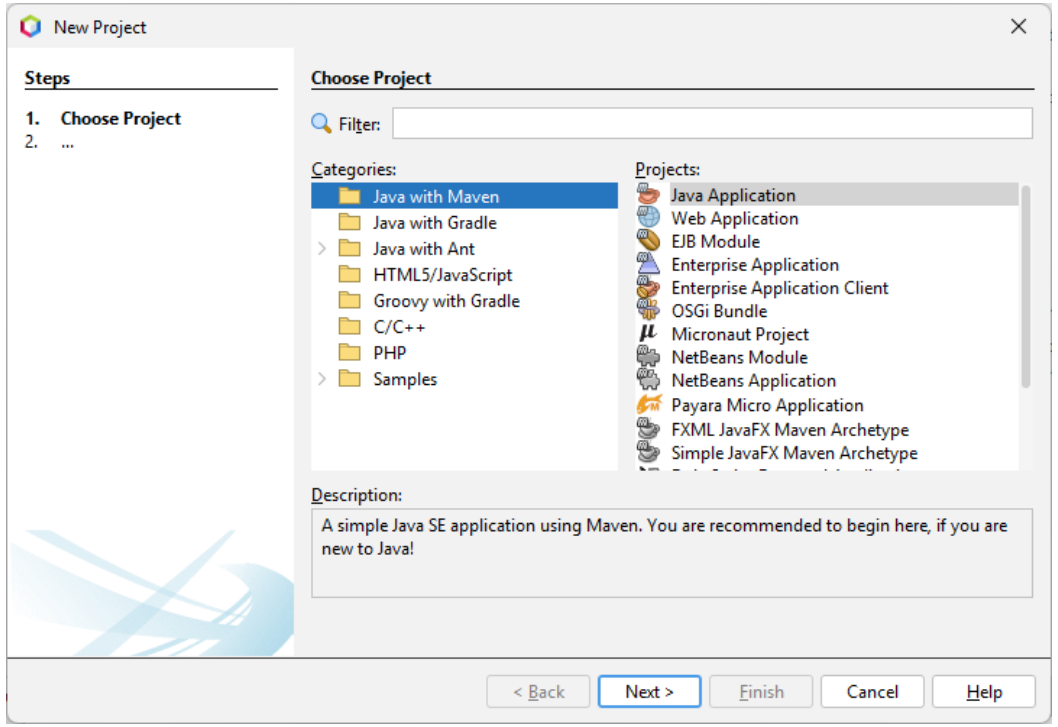
idat

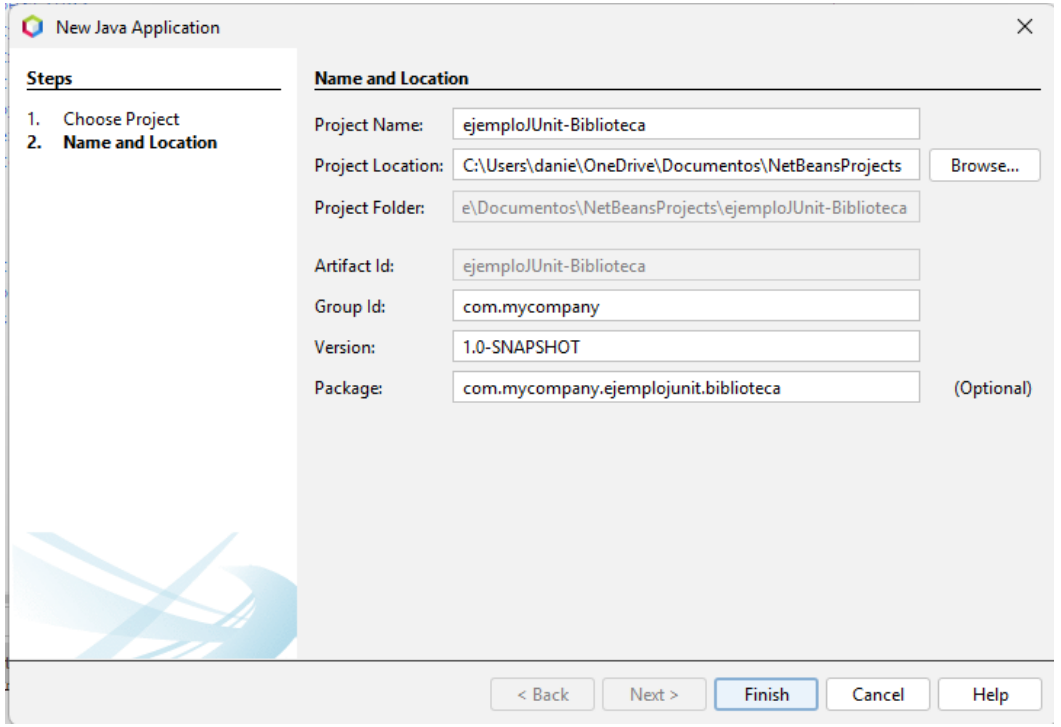
Laboratorio: JUnit

idat

Ejecutar Apache Netbeans

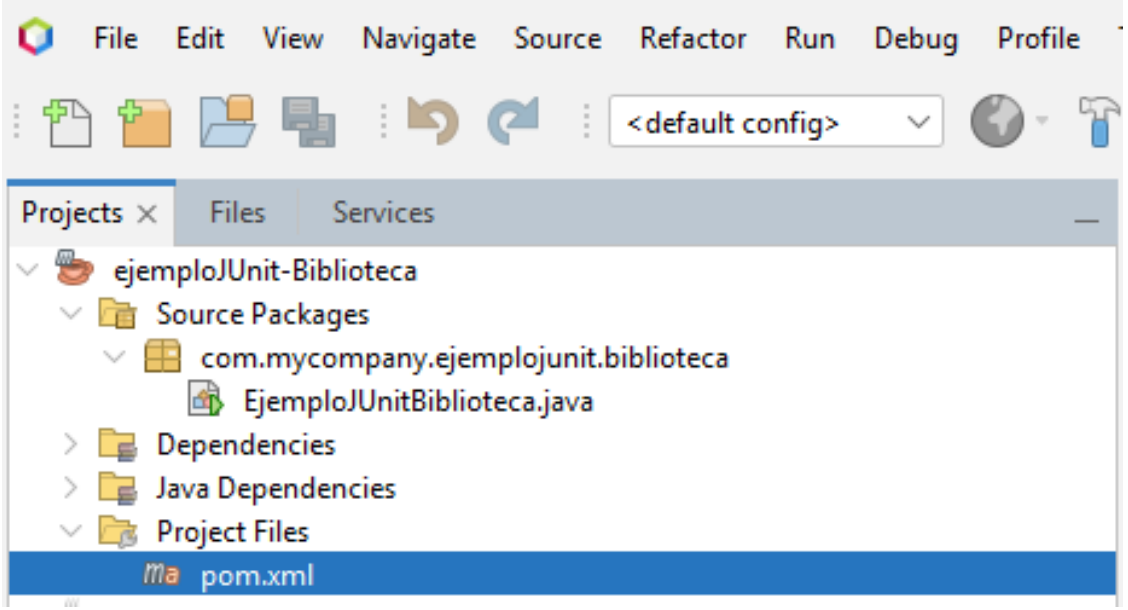
Creamos el Proyecto “ejemploJUnit-Biblioteca”





idat

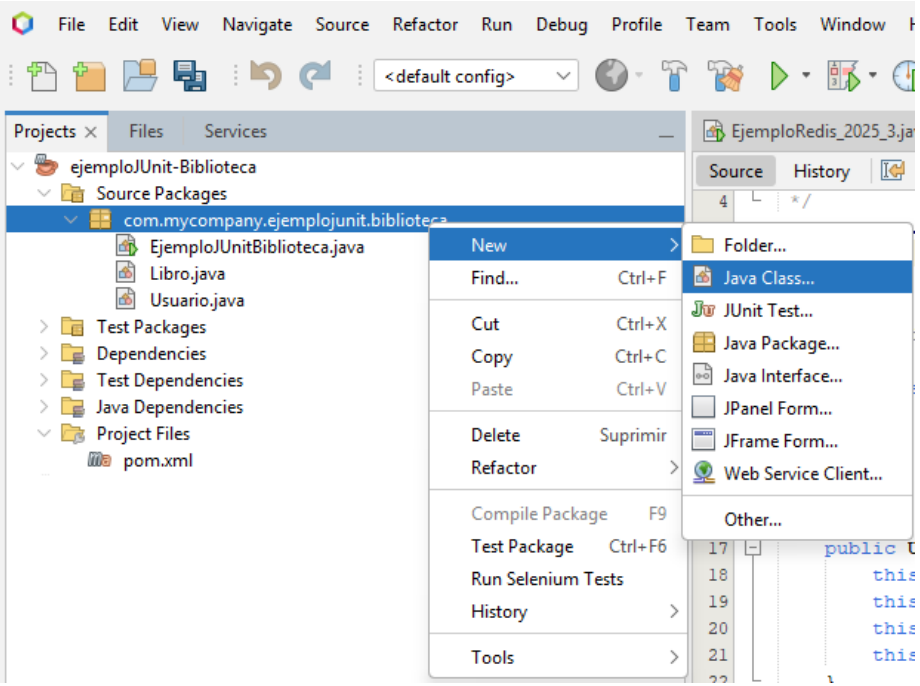
Modificar el pom.xml

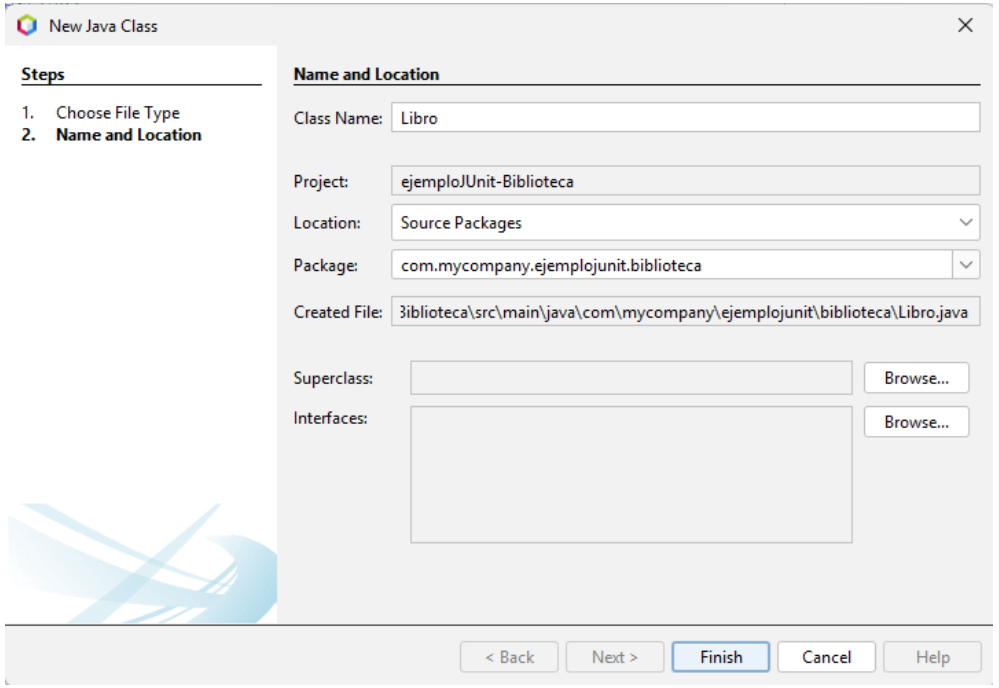



```
<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.14.0</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

idat

Creamos la Clase: Libro





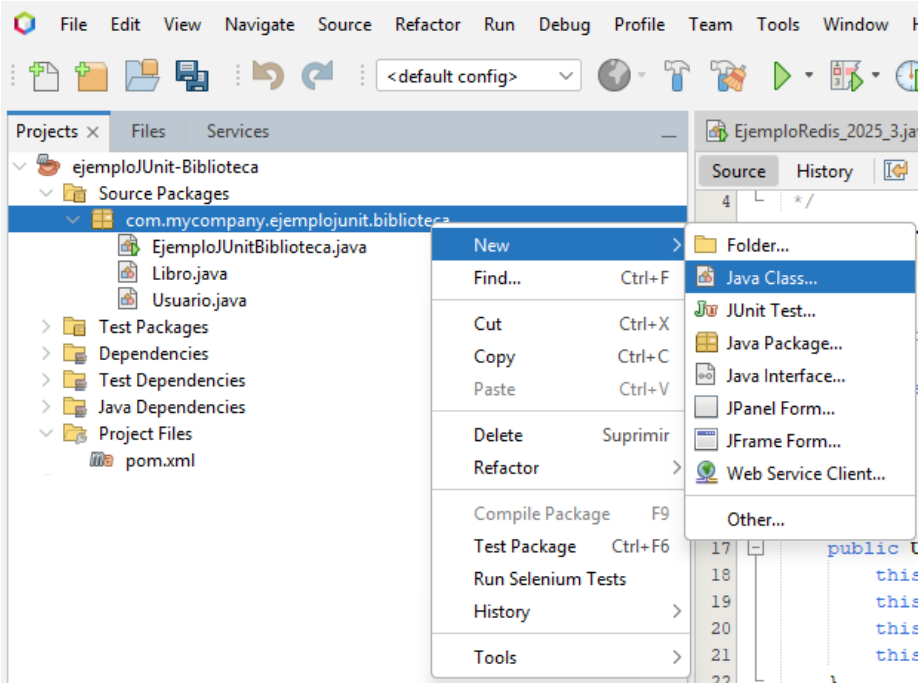
idat

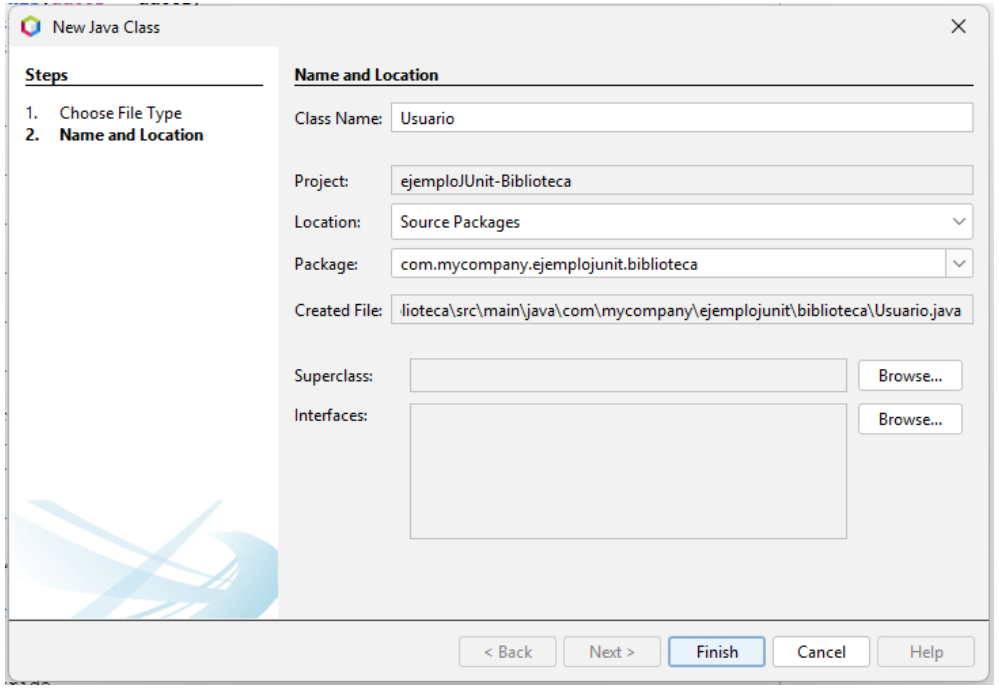
Codificamos la Clase: Libro

```java
public class Libro {
    private String isbn;
    private String titulo;
    private String autor;
    private int anoPublicacion;
    private boolean disponible;

    public Libro(String isbn, String titulo, String autor, int anoPublicacion) {
        this.isbn = isbn;
        this.titulo = titulo;
        this.autor = autor;
        this.anoPublicacion = anoPublicacion;
        this.disponible = true;
    }

    public String getIsbn() { return isbn; }

    public String getTitulo() { return titulo; }

    public String getAutor() { return autor; }

    public int getAnoPublicacion() { return anoPublicacion; }

    public boolean isDisponible() { return disponible; }

    public void setDisponible(boolean disponible) { this.disponible = disponible; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;

        if (obj == null || getClass() != obj.getClass()) return false;

        Libro libro = (Libro)obj;

        return isbn.equals(libro.isbn);
    }

    @Override
    public int hashCode() {
        return isbn.hashCode();
    }
}
```

idat

Creamos la Clase: Usuario





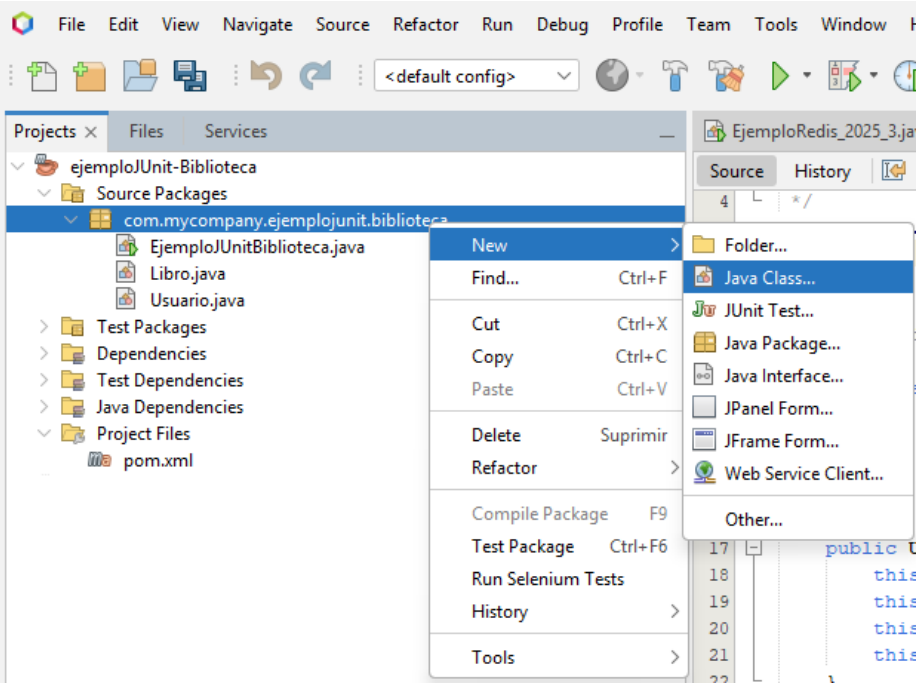
idat

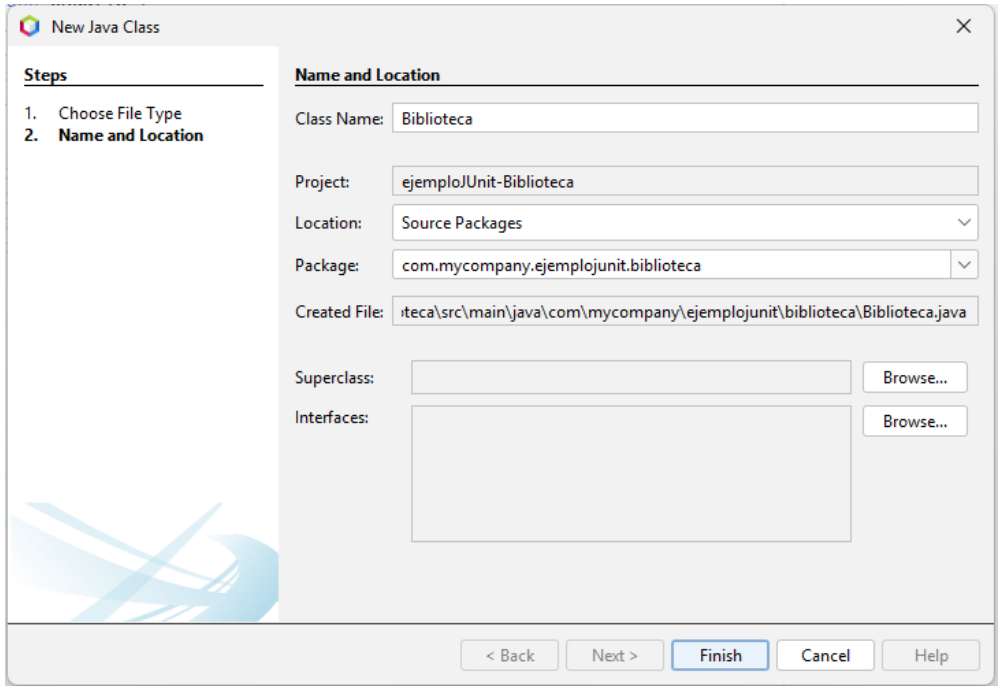
Codificamos la Clase: Usuario

```java
11  public class Usuario {
        private String id;
        private String nombre;
        private String email;
15      private int librosPrestados;
16
17      public Usuario(String id, String nombre, String email) {
18          this.id = id;
19          this.nombre = nombre;
20          this.email = email;
21          this.librosPrestados = 0;
22      }
23
24      public String getId() { return id; }
25
26      public String getNombre() { return nombre; }
27
28      public String getEmail() { return email; }
29
30      public int getLibrosPrestados() { return librosPrestados; }
31
32      public void incrementarLibrosPrestados() {
33          librosPrestados++;
34      }
35
36      public void decrementarLibrosPrestados() {
37          if (librosPrestados > 0) {
38              librosPrestados--;
39          }
40      }
41
42      public boolean puedeTomarPrestado() {
43          return librosPrestados < 3;
44      }
45
46  }
```

idat

Creamos la Clase: Biblioteca





idat

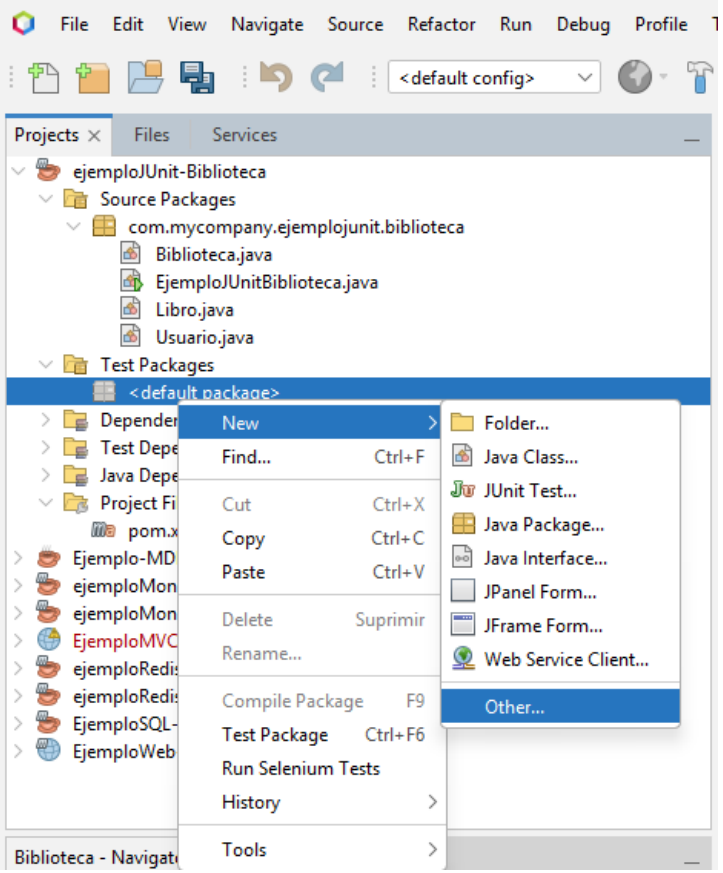
Codificamos la Clase: Biblioteca

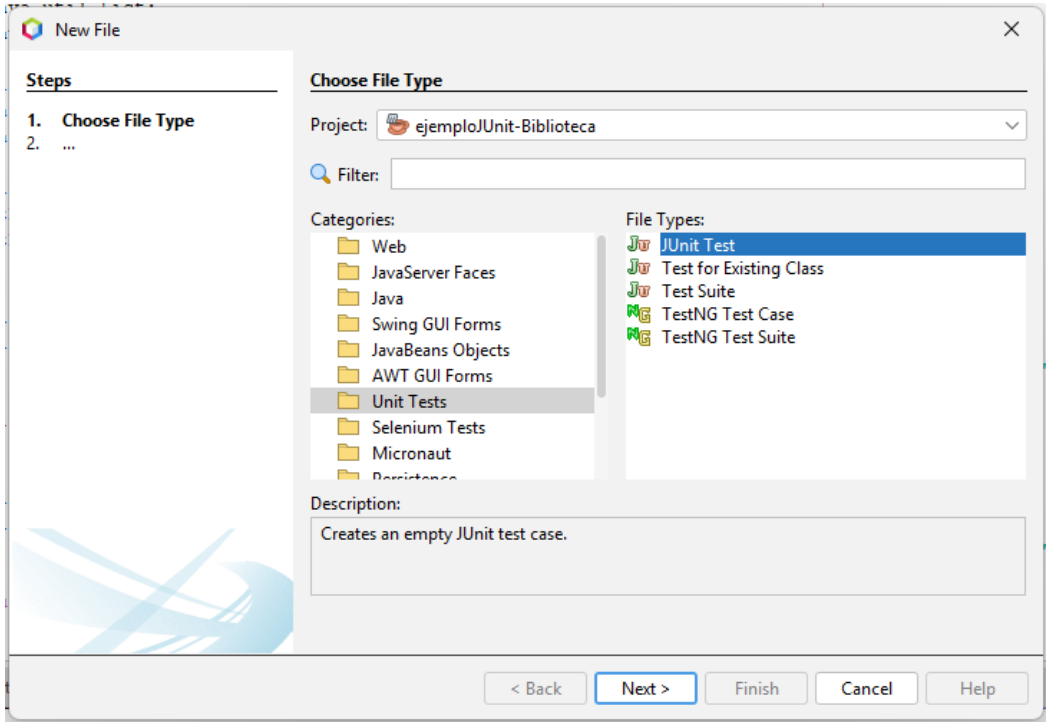
```java
7   import java.util.ArrayList;
8   import java.util.List;
9   import java.util.Optional;
10
11  public class Biblioteca {
12      private List<Libro> libros;
13      private List<Usuario> usuarios;
14
15      public Biblioteca() {
16          this.libros = new ArrayList<>();
17          this.usuarios = new ArrayList<>();
18      }
19
20      public void agregarLibro(Libro libro) {
21          if (buscarLibroPorIsbn(libro.getIsbn()).isPresent()) {
22              throw new IllegalArgumentException("El libro con ISBN " + libro.getIsbn() + " ya existe");
23          }
24          libros.add(libro);
25      }
26
27      public void registrarUsuario(Usuario usuario) {
28          if (buscarUsuarioPorId(usuario.getId()).isPresent()) {
29              throw new IllegalArgumentException("El usuario con ID " + usuario.getId() + " ya existe");
30          }
31          usuarios.add(usuario);
32      }
33
34      public boolean prestarLibro(String isbn, String usuarioId) {
35          Optional<Libro> libroOpt = buscarLibroPorIsbn(isbn);
36          Optional<Usuario> usuarioOpt = buscarUsuarioPorId(usuarioId);
37
38          if (libroOpt.isPresent() && usuarioOpt.isPresent()) {
39              Libro libro = libroOpt.get();
40              Usuario usuario = usuarioOpt.get();
41
42              if (libro.isDisponible() && usuario.puedeTomarPrestado()) {
43                  libro.setDisponible(false);
44                  usuario.incrementarLibrosPrestados();
45                  return true;
46              }
47          }
48          return false;
49      }
```

```java
51      public boolean devolverLibro(String isbn, String usuarioId) {
52          Optional<Libro> libroOpt = buscarLibroPorIsbn(isbn);
53          Optional<Usuario> usuarioOpt = buscarUsuarioPorId(usuarioId);
54
55          if (libroOpt.isPresent() && usuarioOpt.isPresent()) {
56              Libro libro = libroOpt.get();
57              Usuario usuario = usuarioOpt.get();
58
59              if (!libro.isDisponible()) {
60                  libro.setDisponible(true);
61                  usuario.decrementarLibrosPrestados();
62                  return true;
63              }
64          }
65          return false;
66      }
67
68      public Optional<Libro> buscarLibroPorIsbn(String isbn) {
69          return libros.stream()
70                  .filter(libro -> libro.getIsbn().equals(isbn))
71                  .findFirst();
72      }
73
74      public Optional<Usuario> buscarUsuarioPorId(String id) {
75          return usuarios.stream()
76                  .filter(usuario -> usuario.getId().equals(id))
77                  .findFirst();
78      }
79
80      public List<Libro> getLibrosDisponibles() {
81          return libros.stream()
82                  .filter(Libro::isDisponible)
83                  .toList();
84      }
85
86      public int getTotalLibros() {
87          return libros.size();
88      }
89
90      public int getTotalUsuarios() {
91          return usuarios.size();
92      }
93
94  }
```

idat

Creamos la Clase: BibliotecaTest





idat

Codificamos la Clase: BibliotecaTest

```java
package com.mycompany.ejemplojunit.biblioteca;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import static org.junit.jupiter.api.Assertions.*;

/**
 *
 * @author danie
 */
public class BibliotecaTest {
    private Biblioteca biblioteca;
    private Libro libro1;
    private Libro libro2;
    private Usuario usuario1;
    private Usuario usuario2;

    @BeforeEach
    void setUp() {
        biblioteca = new Biblioteca();

        libro1 = new Libro("ISBN-001", "Cien años de soledad", "Gabriel García Márquez", 1967);
        libro2 = new Libro("ISBN-002", "1984", "George Orwell", 1949);

        usuario1 = new Usuario("U001", "Ana García", "ana@email.com");
        usuario2 = new Usuario("U002", "Carlos López", "carlos@email.com");
    }

    @Test
    @DisplayName("Agregar libro a la biblioteca")
    void testAgregarLibro() {
        biblioteca.agregarLibro(libro1);

        assertEquals(1, biblioteca.getTotalLibros());
        assertTrue(biblioteca.buscarLibroPorIsbn("ISBN-001").isPresent());
    }
```

idat

Codificamos la Clase: BibliotecaTest

```java
    @Test
    @DisplayName("No permitir agregar libro con ISBN duplicado")
    void testAgregarLibroDuplicado() {
        biblioteca.agregarLibro(libro1);

        IllegalArgumentException exception = assertThrows(IllegalArgumentException.class,
            () -> biblioteca.agregarLibro(libro1));

        assertEquals("El libro con ISBN ISBN-001 ya existe", exception.getMessage());
    }

    @Test
    @DisplayName("Registrar usuario en la biblioteca")
    void testRegistrarUsuario() {
        biblioteca.registrarUsuario(usuario1);

        assertEquals(1, biblioteca.getTotalUsuarios());
        assertTrue(biblioteca.buscarUsuarioPorId("U001").isPresent());
    }

    @Test
    @DisplayName("Préstamo exitoso de libro")
    void testPrestarLibroExitoso() {
        // Arrange
        biblioteca.agregarLibro(libro1);
        biblioteca.registrarUsuario(usuario1);

        // Act
        boolean resultado = biblioteca.prestarLibro("ISBN-001", "U001");

        // Assert
        assertTrue(resultado);
        assertFalse(libro1.isDisponible());
        assertEquals(1, usuario1.getLibrosPrestados());
    }
```

idat

Codificamos la Clase: BibliotecaTest

```java
    @Test
    @DisplayName("Préstamo fallido - libro no disponible")
    void testPrestarLibroNoDisponible() {
        biblioteca.agregarLibro(libro1);
        biblioteca.registrarUsuario(usuario1);
        biblioteca.registrarUsuario(usuario2);

        // Primer préstamo exitoso
        biblioteca.prestarLibro("ISBN-001", "U001");

        // Segundo préstamo del mismo libro debería fallar
        boolean resultado = biblioteca.prestarLibro("ISBN-001", "U002");

        assertFalse(resultado);
        assertEquals(1, usuario1.getLibrosPrestados());
        assertEquals(0, usuario2.getLibrosPrestados());
    }

    @Test
    @DisplayName("Devolución exitosa de libro")
    void testDevolverLibroExitoso() {
        biblioteca.agregarLibro(libro1);
        biblioteca.registrarUsuario(usuario1);

        // Primero prestamos el libro
        biblioteca.prestarLibro("ISBN-001", "U001");

        // Luego lo devolvemos
        boolean resultado = biblioteca.devolverLibro("ISBN-001", "U001");

        assertTrue(resultado);
        assertTrue(libro1.isDisponible());
        assertEquals(0, usuario1.getLibrosPrestados());
    }
```

idat

Codificamos la Clase: BibliotecaTest

```java
    @Test
    @DisplayName("Usuario no puede tomar más de 3 libros")
    void testLimitePrestamosUsuario() {
        biblioteca.registrarUsuario(usuario1);

        // Agregar 4 libros
        for (int i = 1; i <= 4; i++) {
            Libro libro = new Libro("ISBN-00" + i, "Libro " + i, "Autor " + i, 2000 + i);
            biblioteca.agregarLibro(libro);
        }

        // Prestar 3 libros (debería funcionar)
        for (int i = 1; i <= 3; i++) {
            boolean resultado = biblioteca.prestarLibro("ISBN-00" + i, "U001");
            assertTrue(resultado);
        }

        // Intentar prestar el cuarto libro (debería fallar)
        boolean cuartoPrestamo = biblioteca.prestarLibro("ISBN-004", "U001");
        assertFalse(cuartoPrestamo);
        assertEquals(3, usuario1.getLibrosPrestados());
    }

    @ParameterizedTest
    @DisplayName("Búsqueda de libros por ISBN")
    @CsvSource({
        "ISBN-001, true",
        "ISBN-999, false",
        "ISBN-002, true"
    })
    void testBuscarLibroPorIsbn(String isbn, boolean esperado) {
        biblioteca.agregarLibro(libro1);
        biblioteca.agregarLibro(libro2);

        boolean encontrado = biblioteca.buscarLibroPorIsbn(isbn).isPresent();
        assertEquals(esperado, encontrado);
    }
```

idat

Codificamos la Clase: BibliotecaTest

```java
    @Test
    @DisplayName("Obtener lista de libros disponibles")
    void testGetLibrosDisponibles() {
        biblioteca.agregarLibro(libro1);
        biblioteca.agregarLibro(libro2);
        biblioteca.registrarUsuario(usuario1);

        // Prestar un libro
        biblioteca.prestarLibro("ISBN-001", "U001");

        // Solo debería haber un libro disponible
        assertEquals(1, biblioteca.getLibrosDisponibles().size());
        assertEquals("ISBN-002", biblioteca.getLibrosDisponibles().get(0).getIsbn());
    }

    @Test
    @DisplayName("Verificar estado inicial de usuario")
    void testEstadoInicialUsuario() {
        assertTrue(usuario1.puedeTomarPrestado());
        assertEquals(0, usuario1.getLibrosPrestados());
        assertEquals("Ana García", usuario1.getNombre());
        assertEquals("ana@email.com", usuario1.getEmail());
    }

    @Test
    @DisplayName("Verificar igualdad de libros por ISBN")
    void testIgualdadLibros() {
        Libro mismoLibro = new Libro("ISBN-001", "Otro título", "Otro autor", 2000);

        assertEquals(libro1, mismoLibro);
        assertEquals(libro1.hashCode(), mismoLibro.hashCode());
    }

}
```

idat

¿Dudas?

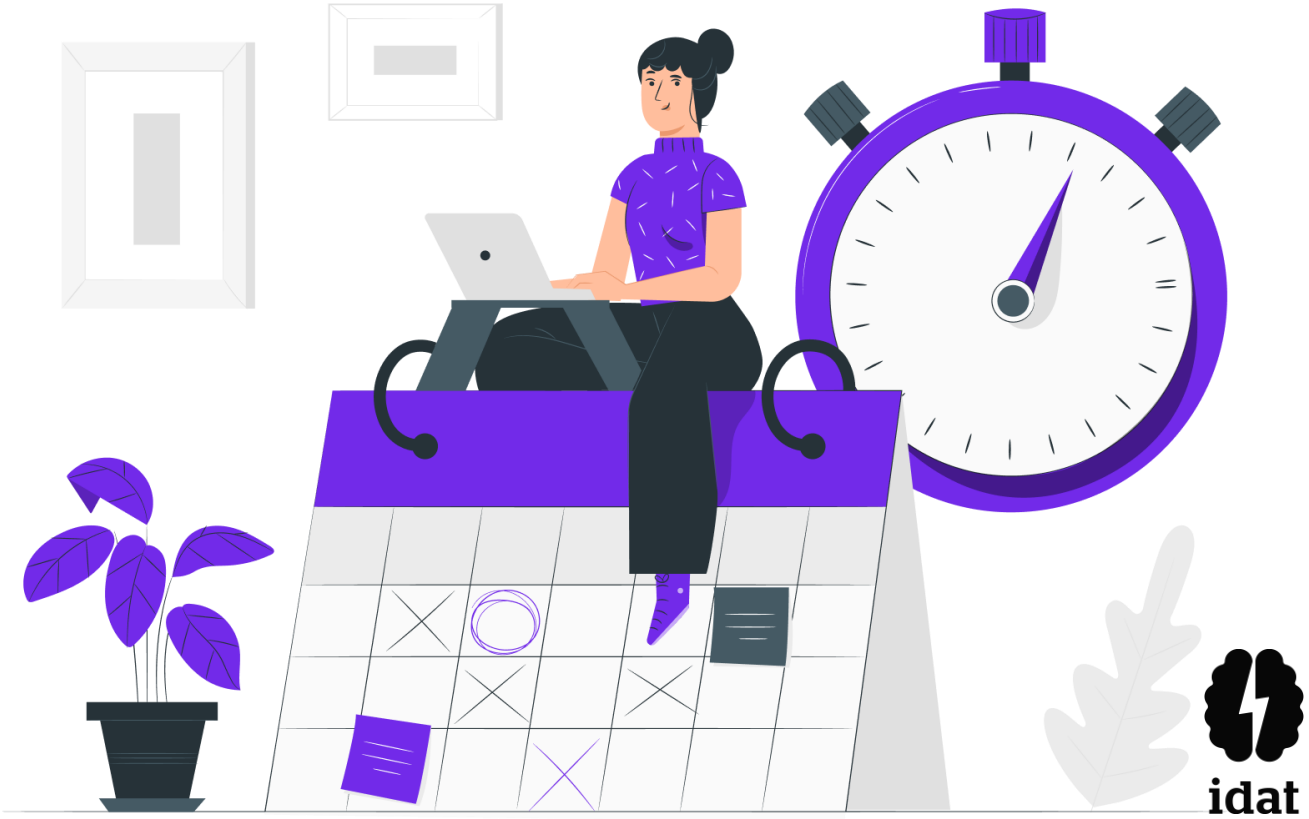
¿Miedos?

¿Temores?

idat

Aplicamos lo aprendido: Asignación para la clase siguiente

Resolver la actividad en el EVA



idat

idat

GRACIAS